DRESSAPP

Your AI Fashion Editor & Wardrobe Ecosystem

Comprehensive User Manual

Technical Integration & User Reference Guide

Version: 1.0 (Phase 6 / Current Production)

Runtime: FARM Stack (FastAPI + React + MongoDB)

Date: July 2026

1. System Overview & Technology Stack

DressApp is an advanced, AI-driven personal wardrobe manager, styling advisor, and garment marketplace. It empowers users to photograph their clothing items, automatically crop and tag them using machine learning, receive weather- and calendar-aware outfit recommendations, scan EU Digital Product Passports (DPP), and list items on an integrated region-aware marketplace.

Core Architecture & Tech Stack

DressApp utilizes the FARM Stack (FastAPI, React, MongoDB) combined with a local Python Machine Learning vision pipeline and multimodal Generative AI capabilities:

- Backend: FastAPI (Python 3.11) with Motor asynchronous driver for MongoDB Atlas database connectivity.
- Frontend: React 19 Single Page Application (SPA) with Tailwind CSS, Shadcn UI elements, and react-i18next for 12 locales support.
- Vision Pipeline: Local SegFormer (HuggingFace) for semantic clothing parsing and rembg (U2-Net) for background removal.
- Generative AI Agent: Google Gemini 2.5 Flash/Pro APIs for conversational styling, detailed tagging, and audio modulations.
- Audio Subsystem: Dual-path Speech-to-Text (STT) and Text-to-Speech (TTS) using browser Web Speech APIs and offline local Piper ONNX support.

2. Prerequisites & Environment Setup

Before deploying or using the DressApp system, ensure that the following requirements are met:

Server & Host Environment

- Docker & Docker Compose: For standard multi-container VPS deployments (4 GB+ RAM recommended for CPU-local model workloads).
- Python Runtime: Python 3.11 with virtualenv manager.
- API Keys: Google Gemini API key (GEMINI_API_KEY) and access credentials for Google OAuth, PayPal REST API, and OpenWeatherMap.

Client Prerequisites

- Browser Permissions: Access to Camera (for live garment capture and DPP QR scanning) and Microphone (for voice commands).

- Modern Web Browsers: Chrome or Safari are recommended for client-side SpeechRecognition (Web Speech API) compatibility.

3. Step-by-Step Instructions & Workflows

3.1 Ingestion Pipeline: Adding Items

The Ingestion Pipeline is the primary way users load items into their digital closet. Users can use traditional camera/uploads, paste digital receipts, or scan EU-compliant DPP QR codes.

To ingest items via Camera or File Upload:

1. Navigate to the 'Add Item' screen in the application.
2. Choose 'Take Photo' (rear camera trigger) or click 'Upload Photos' to open the file selection explorer.
3. The system automatically performs local SHA-256 and horizontal difference-hashing (dHash) in the browser to scan for duplicate garments within the user's closet (~100-180ms).
4. If duplicates are identified, the 'Duplicate Preflight Dialog' appears allowing the user to skip or force ingestion. Batch mode (>5 items) drops duplicates automatically.
5. Once validated, the app initiates a progressive NDJSON stream from the backend. The 'detect' frame displays segmented placeholders within 5-7 seconds, allowing the user to review items while tagging completes.
6. Correct or refine the auto-detected fields (name, category, colors, materials, patterns, etc.). Adjusting the category triggers the SegFormer segmenter to update the cutout mask dynamically.
7. Click 'Save' to commit items instantly. Pre-generated WebP thumbnails resolve instantly on the Closet grid.

To import details via EU Digital Product Passport (DPP):

1. Click the 'Scan QR (DPP)' button on the Add Item page.
2. Grant camera permission. Target the QR code printed on the garment tag, or switch to the 'File' tab to upload a screenshot of the QR.
3. The backend parses the passport URL, resolves the DNS, and performs server-side safety checks (blocking private network loopbacks).
4. The parser downloads the passport content (HTML/JSON-LD), discovers the Product schemas, and normalizes fields like fibre composition, materials, care instructions, and carbon footprint.
5. Verify the extracted information rendered in the green 'Verified DPP Data' box and click 'Save'.

To ingest via Digital Receipts:

1. Switch to the 'Digital Import' tab on the Add Item page.
2. Choose your sub-mode: Paste Text, Upload Image, Upload PDF, or input a Web Link.
3. The parser extracts brand, cost, size, and category facts. Extracted properties are marked as receipt-locked to protect them from future visual re-analysis.

3.2 Conversational AI Stylist & Audio System

The AI Stylist provides conversational fashion guidance. It supports hands-free audio loop communication integrating text-to-speech and speech-to-text workflows.

How to use the Voice Chat Interface:

1. Open the 'AI Stylist' chat page.
2. Tap the microphone icon [Microphone] in the stylist message input bar.
3. Speak your query (e.g., 'What should I wear for a rainy afternoon business meeting?'). The browser Web Speech API transcribes the text live in the input field.
4. If the browser lacks speech recognition, the client records your audio as a WebM binary and sends it to the backend.
5. The backend STT service routes the audio query to the local Gemma4 container. If offline, it automatically falls back to Gemini 2.5 Flash's multimodal transcription.
6. The AI Stylist queries your digital closet and current weather metadata, formulating a recommendations text.
7. The backend requests Gemini 2.5 Flash to generate responses directly in AUDIO format using chosen profiles (puck, aoede, charon).
8. The frontend decodes the base64 audio and plays the styling recommendation. Users can click 'Play reply' or 'Replay' to listen again or cancel playback.

3.3 Profile, Preferences, and Subsystem Dependencies

The Profile page serves as the core control panel for DressApp. Configuration fields directly impact the performance, routing, and behavior of downstream modules.

Accordion Section Dependencies & Rationale

1. Photos & Avatar

- Why does it matter?: It provides the visual identity for personalized styling rendering and canvas overlays.
- Subsystem Dependencies: Avatars and reference photos populate the AvatarViewer2D and OutfitAvatarViewer. To prevent database upload failures (MongoDB's 16MB document boundary) and conserve bandwidth, photos are compressed in-browser to a maximum of 1280px at 82% quality.

2. Style Profile (Modest rules, Dress code)

- Why does it matter?: It establishes personal boundaries for recommended outfits, preventing the AI from generating inappropriate style suggestions.
- Subsystem Dependencies: The selected parameters (e.g., Modest clothing constraints) are fed directly into the styling prompts for Gemini 2.5 Flash, filtering matching wardrobe outputs before they are shown.

3. Details (Name, Phone, Occupation)

- Why does it matter?: It customizes the communication tone and routes notification alerts.
- Subsystem Dependencies: The user's name is dynamically parsed into emails and system-level pushes. The phone number serves as a fallback registry for scheduled alerts. The occupation parameter is fed to the stylist LLM to customize proposals (e.g., corporate office vs. remote work templates).

4. Body & Measurements (Height, Weight, Shapes)

- Why does it matter?: It eliminates sizing guesswork, allowing external retail size comparison and accurate virtual layering.
- Subsystem Dependencies: Measurements are queried directly by the Shopping Assistant Chrome Extension content scripts to read size tables on partner websites (like Zara and Asos) and recommend sizes. They also adjust the rendering scale of clothing segments on the Outfit Canvas.

5. Lifestyle (Status, Sex)

- Why does it matter?: It tailors default recommendations and scores content algorithms.
- Subsystem Dependencies: The sex selection directly affects the ranking logic of daily Trend Scout cards. If a news card category mismatches the user's sex, the algorithm applies a -2.0 score penalty, demoting it in the feed.

6. AI Configuration (SaaS keys, edge mode, credits)

- Why does it matter?: It determines billing routing, operational performance, and network offline status.
- Subsystem Dependencies: Routes text/audio generation queries. Standard setups consume DressApp system credits. Inputting personal API keys (Google AI Studio, Anthropic, OpenAI) redirects charges to the user's developer billing accounts. Selecting edge local mode routes queries to the offline Gemma container.

7. Scheduler & Push (Frequency, daily alarm, style focus)

- Why does it matter?: It manages automatic daily style pushes.
- Subsystem Dependencies: Activates APScheduler cron jobs on the FastAPI backend. Every morning, it triggers push notifications via pywebpush using the client's VAPID keys, matching the selected style focus parameters.

8. Google Calendar (OAuth sync, export rules)

- Why does it matter?: It bridges your wardrobe directly with your real-world calendar events.
- Subsystem Dependencies: Authenticates via Google OAuth. The scheduler queries your calendar to identify events, formats outfits, and pushes calendar events straight to your Google Calendar agenda.

9. Location Services (GPS tracking, weather accuracy)

- Why does it matter?: It coordinates weather-appropriate suggestions and local transaction radius filters.
- Subsystem Dependencies: Triggers navigator.geolocation reverse-geocoding. Coordinates are sent to OpenWeatherMap API to adjust the stylist recommendations (e.g., rainwear for downpours). It also calculates distances for local Marketplace listings and experts (e.g., Lisbon radius checks).

10. Voice & Language (Virtual stylist voice selection)

- Why does it matter?: It establishes text dictionary locales and voice modulations.
- Subsystem Dependencies: Controls the active language for translations via react-i18next. The voice selection maps BCP-47 voice codes (e.g., he-IL or ar-JO) to client Web Speech synthesis voices or offline Piper TTS models.

11. Invite Friends (Share payload API)

- Why does it matter?: It provides a viral loop for free closet expansion.
- Subsystem Dependencies: Appends the referrer's MongoDB ID to the URL. New registrations dynamically query this ID and atomically increment the referrer's closet_capacity_bonus by +10 slots, modifying the limit guards in closet.py.

3.4 Wardrobe Insights Dashboard

The Wardrobe Insights dashboard is a personal fashion analytics center designed to help users analyze their wardrobe value, track garment utilization, and cultivate conscious wear habits.

- Closet Worth (Total Value): The cumulative sum of the purchase prices of all items currently saved in the closet.
- Closet Utilization: The percentage of garments in the wardrobe that have been worn at least once.
- Average Cost-per-Wear (CPW): The average efficiency score across all priced items. CPW is calculated as Garment Price / Wear Count.
- Dynamic Distribution Charts: Users can toggle between Color Palette, Materials, and Subcategories breakdown views built with Recharts.
- Efficiency Leaderboard: Displays the Top 5 Most Efficient Items based on the lowest Cost-per-Wear score.

3.5 Outfit Canvas & Planner

Build, layer, and review outfit proposals on an interactive 2D avatar canvas.

- Outerwear Layering (Dual Canvas): If your outfit includes outerwear (e.g., a jacket) over a top, the page renders two vertical canvas modules: 'With Outerwear' and 'Without Outerwear'.
- Interactive 2D Elements: Tap directly on any clothing piece on the avatar's body. The app routes you straight to that garment's detail screen.
- Review Metrics Tab: Click the details button and choose the Metrics tab to view progress bars for compatibility criteria: Color Harmony, Pattern Compatibility, Body Fitting, Weather Match, Event Match, and Location Match.
- Rename/Describe: Click the Pencil icon to edit outfit names and descriptions dynamically.

3.6 Suitcase Assistant

Organize your packing requirements for trips without overpacking.

- Intelligent Curation: Generates daily outfits and packing lists based on trip setup dates, duration, local weather, and calendar events.
- State Retention: Maintains the active suitcase state and history seamlessly, ensuring that progress and refinement notes are not lost.
- Refinement Chat: Chat with the AI stylist to tweak the suitcase while preserving the rest of the curated list.

3.7 Scheduler & Push Reminders

Set daily styling alerts to receive outfit recommendations automatically.

- Job Scheduling: Powered by APScheduler running a recurring cron task inside the FastAPI server.
- Native Push: Uses pywebpush to transmit native browser notifications using VAPID keys.
- Simulated Logging: Logs notifications in the simulated Notification Center on the web app for easy testing.

3.8 Marketplace & circular economy

Engage in the peer-to-peer circular fashion marketplace.

- Create a Listing: Open an item's detail page, select Edit Intent, and choose For Sale (input price/currency), Rent (input daily tariff), Swap (mark open for trade), or Donate (publish free).
- Zero-Latency Caching: Synced globally via transactionsStore. Try-On Sandbox allows renters/buyers to test-fit a listing against items in their private closet before checking out.
- PayPal Integration: direct PayPal Checkout secures payment, payouts, and captures platform fees (7% Platform Commission).

3.9 Admin Panel Dashboard

System liveness validation, financial bookkeeping, and user account management.

- Overview Tab: Audit Gross Volumes and Platform Fee income. Inspect Provider Activity Table to view downstream statistics.
- Providers Tab: Click Verify Key to ping the Gemini API. Toggle Eyes Vision Override to route image analysis between Gemini and Gemma.
- Users Tab: View roles, credits usage, and role management (Promote/Demote admin role toggles).
- Listings Tab: Pause or activate listings.

4. Expected Results

Upon successfully completing core interactions, you should expect the following results:

- Garment Cutouts: Garment cutouts should be clean, borderless transparent PNG files.
- Inpainted Garments: Garments cropped at photo edges display a 'useReconstructed' toggle that renders the AI-inpainted (Nano Banana) complete item shape.
- Closet Capacity Expansion: Confirm limits increased from 150 (e.g. showing Pro or 160+ items) after active subscriptions or referral signups.
- DPP Verified Badge: Successfully scanned products show a green 'Verified DPP Data' box containing the details.
- Stylist Audio Waveform: Audio suggestions will auto-play with visual waveform feedback.

5. Troubleshooting

- HTTP 402 Payment Required: Appears when adding items beyond the 150 slot limit. Solve by subscribing to Pro or sharing your referral invite link.
- SSRF / DNS Parse Failure on DPP: Ensure the scanned QR URL points to a public domain. Private network IPs are blocked by the backend.
- Camera Access Blocked: Check browser permissions. If blocked, switch to 'File Upload' mode inside the DPP Scanner to select screenshots.
- Voice Chat Failure / Silence: Primary Web Speech recognition requires Chrome/Safari. For other browsers, standard typing input remains available.
- OOM Crashes during Batch Uploads: Batches larger than 5 automatically route sequentially. Ensure the server has 4 GB RAM.

6. Limitations

- ML CPU Overhead: Local SegFormer and rembg run CPU-local on development servers, concurrent background analysis processes might introduce CPU spikes.
- Receipt Parsing Limits: Handwritten or highly blurred receipts may fail validation.
- Offline Piper TTS: Offline mobile voice synthesis is restricted to local packages.
- Image Size Constraints: Avatar and profile uploads are compressed locally in the browser to 82% quality to fit within the 16MB MongoDB document boundary.